

Layout, Tailwind und die Toolchain

Termin 4 · 20.10.2026

FÄLLIG

bis Termin 5 am 27.10.

ZEITBEDARF

ca. 2,5 Stunden

BAUSTEINE IM REPO

t4-flexbox-grid, t4-responsive-karten, t4-tailwind-cdn, t4-vite-tailwind

KONTAKT

dhbw@theaisoftwarecompany.com

Heute wird aus der gestylten Seite ein Layout, das auf dem Handy funktioniert – und ihr lernt, woher der Code kommt, den ihr nicht selbst schreibt.

Teil A · Pflicht

A1 Layout mit Flexbox und Grid

Mindestens **eine** Stelle mit Flexbox (Navigation, Button-Leiste, Kartenkopf) und **eine** mit Grid (Übersicht, Seitenraster).

Arbeitet mobile first: erst die schmale Ansicht, dann mit `min-width` erweitern. Oder nehmt `repeat(auto-fit, minmax(..., 1fr))` und kommt ganz ohne Media Query aus.

→ **Fertig, wenn** die Seite bei 360 px, 768 px und 1280 px jeweils sinnvoll aussieht – geprüft im Device Mode, nicht geraten.

A2 Tailwind einbinden

Bindet Tailwind ein (Play CDN genügt für diese Woche) und stellt **eine** Seite oder **einen** größeren Abschnitt auf Utility-Klassen um.

Nicht alles. Der Vergleich ist die Aufgabe: dieselbe Karte einmal mit eigenem CSS, einmal mit Tailwind.

→ **Fertig, wenn** der umgestellte Teil genauso aussieht wie vorher und ihr benennen könnt, welche Tailwind-Klasse welcher CSS-Eigenschaft entspricht.

A3 Drei Komponenten

Sucht die drei Bausteine, die sich in eurem Projekt wiederholen – typisch: Button, Karte, Navbar. Macht sie **konsistent**: gleiche Abstände, gleiche Radien, gleiche Farben.

Schreibt in die README, welche drei es sind und wo sie vorkommen.

→ **Fertig, wenn** an keiner Stelle mehr zwei Buttons unterschiedlich aussehen, die dasselbe tun.

A4 npm und ein Build

Richtet im Projekt eine Toolchain ein:

```
npm create vite@latest
npm install
npm run dev
npm run build
```

Schaut danach in `dist/` und in `package.json` .

`package-lock.json` gehört ins Repo, `node_modules/` gehört in `.gitignore` .

→ **Fertig, wenn** `npm run build` durchläuft, ihr sagen könnt, was in `dist/` liegt, und `git status` **kein** `node_modules` zeigt.

Teil B · Bonus (+1 Punkt)

Stellt euer Projekt vollständig auf Vite mit dem Tailwind-Build um (statt Play CDN) und schaut euch die erzeugte CSS-Datei in `dist/assets/` an.

In die README: wie groß die Datei ist und warum sie so klein bleibt, obwohl Tailwind tausende Klassen kennt.

Teil C · Verständnisfragen

1. Was bedeutet die Einheit `fr` in `grid-template-columns: 1fr 3fr` ?
2. Was genau macht `npm run build` mit eurem Projektordner – und warum ladet ihr danach nicht den ganzen Ordner auf den Server?
3. Warum gehört `node_modules/` nicht ins Repo, `package-lock.json` aber schon?

So arbeitest du hier mit KI

Diese Woche lernt ihr die wichtigste Grenze kennen: **Modelle kennen den Stand ihres Trainings, nicht den von heute**. Tailwind 4 unterscheidet sich vom Setup für Tailwind 3, und ihr werdet Antworten bekommen, die zur alten Version gehören.

Deshalb:

„Ich benutze Tailwind Version 4 mit Vite. Hier ist meine `package.json` und meine `vite.config.js` . Die Doku sagt . Prüfe mein Setup gegen diese Angaben.“

Und bei jedem Paketvorschlag:

„Nenne mir die npm-Seite zu diesem Paket, die Downloads pro Woche und das Datum der letzten Version.“

Dann nachschauen. Es kommt vor, dass Modelle Pakete erfinden – und es gibt Leute, die genau solche Namen registrieren und Schadcode hineinlegen.